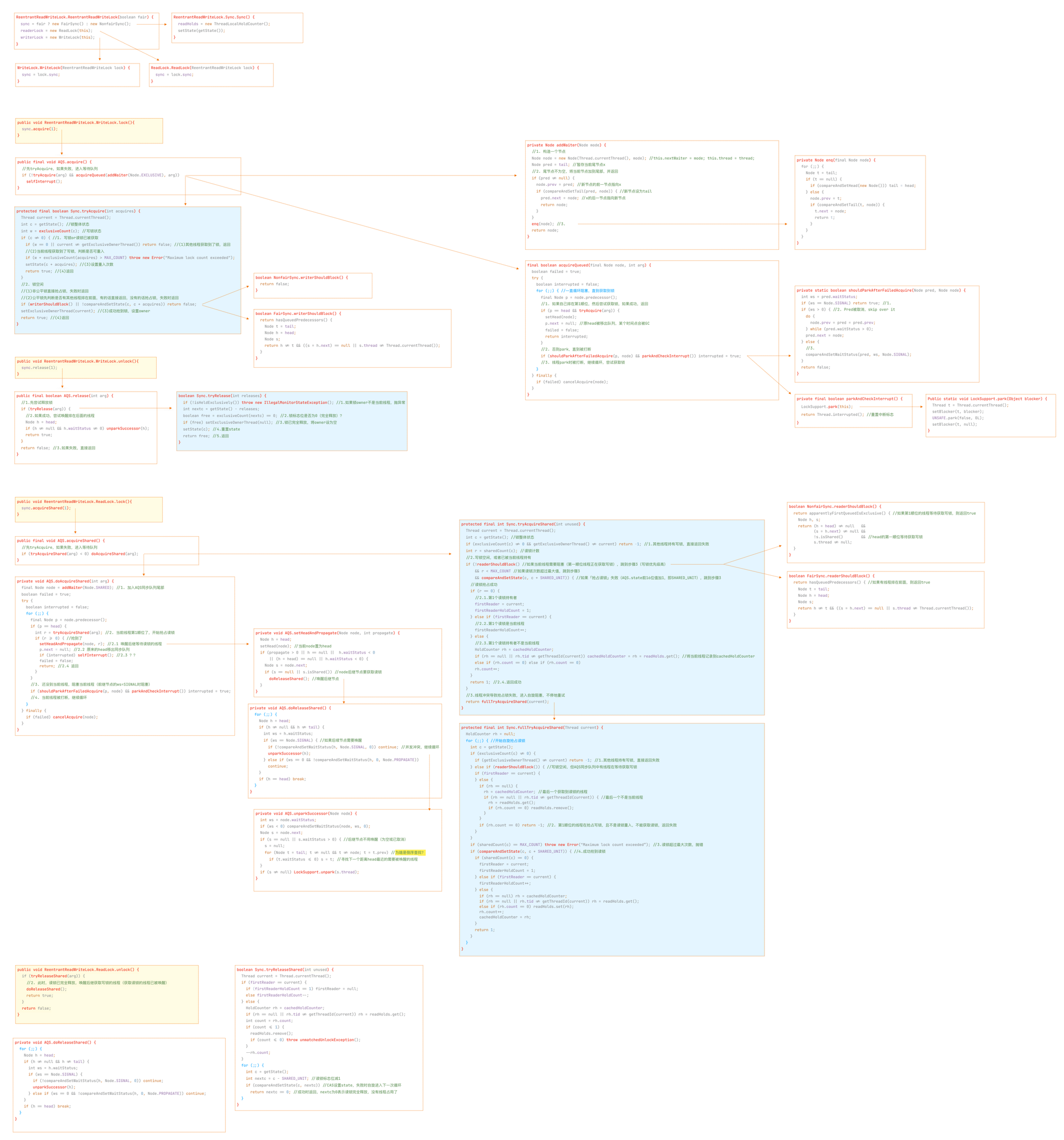

```
ReentrantReadWriteLock.ReentrantReadWriteLock(boolean fair) {
  sync = fair ? new FairSync() : new NonfairSync();
  readerLock = new ReadLock(this);
  writerLock = new WriteLock(this);
}
```

```
ReentrantReadWriteLock.Sync.Sync() {
  readHolds = new ThreadLocalHoldCounter();
  setState(getState());
}
```

```
WriteLock.WriteLock(ReentrantReadWriteLock lock) {
  sync = lock.sync;
}
```

```
ReadLock.ReadLock(ReentrantReadWriteLock lock) {
  sync = lock.sync;
}
```

```
public void ReentrantReadWriteLock.WriteLock.lock(){
  sync.acquire(1);
}
```

```
public final void AQS.acquire() {
  //先tryAcquire，如果失败，进入等待队列
  if (!tryAcquire(arg) && acquireQueued(addWaiter(Node.EXCLUSIVE), arg))
    selfInterrupt();
}
```

```
protected final boolean Sync.tryAcquire(int acquires) {
  Thread current = Thread.currentThread();
  int c = getState(); //锁整体状态
  int w = exclusiveCount(c); //写锁状态
  if (c != 0) { //1. 写锁or读锁已被获取
    if (w == 0 || current != getExclusiveOwnerThread()) return false; //(1)其他线程获取到了锁，返回
    //(2)当前线程获取到了写锁，判断是否可重入
    if (w + exclusiveCount(acquires) > MAX_COUNT) throw new Error("Maximum lock count exceeded");
    setState(c + acquires); //(3)设置重入次数
    return true; //(4)返回
  }
  //2. 锁空闲
  //(1)非公平锁直接抢占锁，失败时返回
  //(2)公平锁先判断是否有其他线程排在前面，有的话直接返回，没有的话抢占锁，失败时返回
  if (writerShouldBlock() || !compareAndSetState(c, c + acquires)) return false;
  setExclusiveOwnerThread(current); //(3)成功抢到锁，设置owner
  return true; //(4)返回
}
```

```
boolean NonfairSync.writerShouldBlock() {
  return false;
}
```

```
boolean FairSync.writerShouldBlock() {
  return hasQueuedPredecessors() {
    Node t = tail;
    Node h = head;
    Node s;
    return h != t && ((s = h.next) == null || s.thread != Thread.currentThread());
  }
}
```

```
private Node addWaiter(Node mode) {
  //1. 构造一个节点
  Node node = new Node(Thread.currentThread(), mode); //this.nextWaiter = mode; this.thread = thread;
  Node pred = tail; //暂存当前尾节点x
  //2. 尾节点不为空，将当前节点加到尾部，并返回
  if (pred != null) {
    node.prev = pred; //新节点的前一节点指向x
    if (compareAndSetTail(pred, node)) { //新节点设为tail
      pred.next = node; //x的后一节点指向新节点
      return node;
    }
  }
  enq(node); //3.
  return node;
}
```

```
private Node enq(final Node node) {
  for (;;) {
    Node t = tail;
    if (t == null) {
      if (compareAndSetHead(new Node())) tail = head;
    } else {
      node.prev = t;
      if (compareAndSetTail(t, node)) {
        t.next = node;
        return t;
      }
    }
  }
}
```

```
final boolean acquireQueued(final Node node, int arg) {
  boolean failed = true;
  try {
    boolean interrupted = false;
    for (;;) { //一直循环阻塞，直到获取到锁
      final Node p = node.predecessor();
      //1. 如果自己排在第1顺位，然后尝试获取锁，如果成功，返回
      if (p == head && tryAcquire(arg)) {
        setHead(node);
        p.next = null; //原head被移出队列，某个时间点会被GC
        failed = false;
        return interrupted;
      }
      //2. 否则park，直到被打断
      if (shouldParkAfterFailedAcquire(p, node) && parkAndCheckInterrupt()) interrupted = true;
      //3. 线程park时被打断，继续循环，尝试获取锁
    }
  } finally {
    if (failed) cancelAcquire(node);
  }
}
```

```
private static boolean shouldParkAfterFailedAcquire(Node pred, Node node) {
  int ws = pred.waitStatus;
  if (ws == Node.SIGNAL) return true; //1.
  if (ws > 0) { //2. Pred被取消, skip over it
    do {
      node.prev = pred = pred.prev;
    } while (pred.waitStatus > 0);
    pred.next = node;
  } else {
    //3.
    compareAndSetWaitStatus(pred, ws, Node.SIGNAL);
  }
  return false;
}
```

```
private final boolean parkAndCheckInterrupt() {
  LockSupport.park(this);
  return Thread.interrupted(); //重置中断标志
}
```

```
Public static void LockSupport.park(Object blocker) {
  Thread t = Thread.currentThread();
  setBlocker(t, blocker);
  UNSAFE.park(false, 0L);
  setBlocker(t, null);
}
```

```
public void ReentrantReadWriteLock.WriteLock.unlock(){
  sync.release(1);
}
```

```
public final boolean AQS.release(int arg) {
  //1.先尝试释放锁
  if (tryRelease(arg)) {
    //2.如果成功，尝试唤醒排在后面的线程
    Node h = head;
    if (h != null && h.waitStatus != 0) unparkSuccessor(h);
    return true;
  }
  return false; //3.如果失败，直接返回
}
```

```
boolean Sync.tryRelease(int releases) {
  if (!isHeldExclusively()) throw new IllegalMonitorStateException(); //1.如果锁owner不是当前线程，抛异常
  int nextc = getState() - releases;
  boolean free = exclusiveCount(nextc) == 0; //2.锁标志位是否为0（完全释放）？
  if (free) setExclusiveOwnerThread(null); //3.锁已完全释放，将owner设为空
  setState(c); //4.重置state
  return free; //5.返回
}
```

```
public void ReentrantReadWriteLock.ReadLock.lock(){
  sync.acquireShared(1);
}
```

```
public final void AQS.acquireShared() {
  //先tryAcquire，如果失败，进入等待队列
  if (tryAcquireShared(arg) < 0) doAcquireShared(arg);
}
```

```
private void AQS.doAcquireShared(int arg) {
  final Node node = addWaiter(Node.SHARED); //1. 加入AQS同步队列尾部
  boolean failed = true;
  try {
    boolean interrupted = false;
    for (;;) {
      final Node p = node.predecessor();
      if (p == head) {
        int r = tryAcquireShared(arg); //2. 当前线程第1顺位了，开始抢占读锁
        if (r >= 0) { //抢到了
          setHeadAndPropagate(node, r); //2.1 唤醒后继等待读锁的线程
          p.next = null; //2.2 原来的head移出同步队列
          if (interrupted) selfInterrupt(); //2.3 ? ?
          failed = false;
          return; //2.4 返回
        }
      }
      //3. 还没到当前线程，阻塞当前线程（前继节点的ws=SIGNAL时阻塞）
      if (shouldParkAfterFailedAcquire(p, node) && parkAndCheckInterrupt()) interrupted = true;
      //4. 当前线程被打断，继续循环
    }
  } finally {
    if (failed) cancelAcquire(node);
  }
}
```

```
private void AQS.setHeadAndPropagate(Node node, int propagate) {
  Node h = head;
  setHead(node); //当前node置为head
  if (propagate > 0 || h == null ||  h.waitStatus < 0
      || (h = head) == null || h.waitStatus < 0) {
    Node s = node.next;
    if (s == null || s.isShared()) //node后继节点要获取读锁
      doReleaseShared(); //唤醒后继节点
  }
}
```

```
private void AQS.doReleaseShared() {
  for (;;) {
    Node h = head;
    if (h != null && h != tail) {
      int ws = h.waitStatus;
      if (ws == Node.SIGNAL) { //如果后续节点需要唤醒
        if (!compareAndSetWaitStatus(h, Node.SIGNAL, 0)) continue; //并发冲突，继续循环
        unparkSuccessor(h);
      } else if (ws == 0 && !compareAndSetWaitStatus(h, 0, Node.PROPAGATE))
        continue;
    }
    if (h == head) break;
  }
}
```

```
private void AQS.unparkSuccessor(Node node) {
  int ws = node.waitStatus;
  if (ws < 0) compareAndSetWaitStatus(node, ws, 0);
  Node s = node.next;
  if (s == null || s.waitStatus > 0) { //后继节点不用唤醒（为空或已取消）
    s = null;
    for (Node t = tail; t != null && t != node; t = t.prev) //为啥是倒序查找?
      if (t.waitStatus <= 0) s = t; //寻找下一个距离head最近的需要被唤醒的线程
  }
  if (s != null) LockSupport.unpark(s.thread);
}
```

```
protected final int Sync.tryAcquireShared(int unused) {
  Thread current = Thread.currentThread();
  int c = getState(); //锁整体状态
  if (exclusiveCount(c) != 0 && getExclusiveOwnerThread() != current) return -1; //1.其他线程持有写锁，直接返回失败
  int r = sharedCount(c); //读锁计数
  //2.写锁空闲，或者已被当前线程持有
  if (!readerShouldBlock() //如果当前线程需要阻塞（第一顺位线程正在获取写锁），跳到步骤3（写锁优先级高）
      && r < MAX_COUNT //如果读锁次数超过最大值，跳到步骤3
      && compareAndSetState(c, c + SHARED_UNIT)) { //如果「抢占读锁」失败（AQS.state前16位值加1，即SHARED_UNIT），跳到步骤3
    //读锁抢占成功
    if (r == 0) {
      //2.1.第1个读锁持有者
      firstReader = current;
      firstReaderHoldCount = 1;
    } else if (firstReader == current) {
      //2.2.第1个读锁是当前线程
      firstReaderHoldCount++;
    } else {
      //2.3.第1个读锁持有者不是当前线程
      HoldCounter rh = cachedHoldCounter;
      if (rh == null || rh.tid != getThreadId(current)) cachedHoldCounter = rh = readHolds.get(); //将当前线程记录到cachedHoldCounter
      else if (rh.count == 0) else if (rh.count == 0)
      rh.count++;
    }
    return 1; //2.4.返回成功
  }
  //3.线程冲突导致抢占锁失败，进入自旋阻塞，不停地重试
  return fullTryAcquireShared(current);
}
```

```
boolean NonfairSync.readerShouldBlock() {
  return apparentlyFirstQueuedIsExclusive() { //如果第1顺位的线程等待获取写锁，则返回true
    Node h, s;
    return (h = head) != null   &&
           (s = h.next) != null &&
           !s.isShared()        && //head的第一顺位等待获取写锁
           s.thread != null;
  }
}
```

```
boolean FairSync.readerShouldBlock() {
  return hasQueuedPredecessors() { //如果有线程排在前面，则返回true
    Node t = tail;
    Node h = head;
    Node s;
    return h != t && ((s = h.next) == null || s.thread != Thread.currentThread());
  }
}
```

```
protected final int Sync.fullTryAcquireShared(Thread current) {
  HoldCounter rh = null;
  for (;;) { //开始自旋抢占读锁
    int c = getState();
    if (exclusiveCount(c) != 0) {
      if (getExclusiveOwnerThread() != current) return -1; //1.其他线程持有写锁，直接返回失败
    } else if (readerShouldBlock()) { //写锁空闲，但AQS同步队列中有线程在等待获取写锁
      if (firstReader == current) {
      } else {
        if (rh == null) {
          rh = cachedHoldCounter; //最后一个获取到读锁的线程
          if (rh == null || rh.tid != getThreadId(current)) { //最后一个不是当前线程
            rh = readHolds.get();
            if (rh.count == 0) readHolds.remove();
          }
        }
        if (rh.count == 0) return -1; //2. 第1顺位的线程在抢占写锁，且不是读锁重入，不能获取读锁，返回失败
      }
    }
    if (sharedCount(c) == MAX_COUNT) throw new Error("Maximum lock count exceeded"); //3.读锁超过最大次数，抛错
    if (compareAndSetState(c, c + SHARED_UNIT)) { //4.成功抢到读锁
      if (sharedCount(c) == 0) {
        firstReader = current;
        firstReaderHoldCount = 1;
      } else if (firstReader == current) {
        firstReaderHoldCount++;
      } else {
        if (rh == null) rh = cachedHoldCounter;
        if (rh == null || rh.tid != getThreadId(current)) rh = readHolds.get();
        else if (rh.count == 0) readHolds.set(rh);
        rh.count++;
        cachedHoldCounter = rh;
      }
      return 1;
    }
  }
}
```

```
public void ReentrantReadWriteLock.ReadLock.unlock() {
  if (tryReleaseShared(arg)) {
    //2. 此时，读锁已完全释放，唤醒后继获取写锁的线程（获取读锁的线程已被唤醒）
    doReleaseShared();
    return true;
  }
  return false;
}
```

```
boolean Sync.tryReleaseShared(int unused) {
  Thread current = Thread.currentThread();
  if (firstReader == current) {
    if (firstReaderHoldCount == 1) firstReader = null;
    else firstReaderHoldCount--;
  } else {
    HoldCounter rh = cachedHoldCounter;
    if (rh == null || rh.tid != getThreadId(current)) rh = readHolds.get();
    int count = rh.count;
    if (count <= 1) {
      readHolds.remove();
      if (count <= 0) throw unmatchedUnlockException();
    }
    --rh.count;
  }
  for (;;) {
    int c = getState();
    int nextc = c - SHARED_UNIT; //读锁标志位减1
    if (compareAndSetState(c, nextc)) //CAS设置state，失败时自旋进入下一次循环
      return nextc == 0; //成功时返回，nextc为0表示读锁完全释放，没有线程占用了
  }
}
```

```
private void AQS.doReleaseShared() {
  for (;;) {
    Node h = head;
    if (h != null && h != tail) {
      int ws = h.waitStatus;
      if (ws == Node.SIGNAL) {
        if (!compareAndSetWaitStatus(h, Node.SIGNAL, 0)) continue;
        unparkSuccessor(h);
      } else if (ws == 0 && !compareAndSetWaitStatus(h, 0, Node.PROPAGATE)) continue;
    }
    if (h == head) break;
  }
}
```